4-bit Array Multiplier: RTL to GDSII on SkyWater SKY130

<NAME> · Roll number <ROLL_NUMBER> · Digital VLSI Design assignment · Repository: https://github.com/GuruSwaD/dvd-assignment-

1 Introduction

This report takes a 4-bit × 4-bit unsigned array multiplier from Verilog RTL to a DRC- and LVS-clean GDSII layout using only open-source tools on the SkyWater 130 nm process with the `sky130_fd_sc_hd` standard-cell library. The design registers both inputs and the 8-bit product, so it has a latency of 2 clock cycles and a throughput of one product per cycle. The flow is: RTL simulation (Icarus Verilog), logic synthesis (Yosys 0.62 + ABC), static timing analysis (OpenSTA 2.7.0, typical corner `tt_025C_1v80`) and physical design with LibreLane 3.0.16 (OpenROAD, Magic, KLayout, Netgen). The tools ran natively on Windows 11 through WSL 2 (Ubuntu 24.04) from LibreLane's Nix environment instead of the IIC-OSIC-TOOLS Docker image; the tool set and the PDK (open_pdks `8afc834`) are the same.

2 Design and verification (Part A)

Design. `full_adder` uses two `assign` statements: *sum* = a ⊕ b ⊕ cin and *cout* = a·b + cin·(a ⊕ b). `mult_array` registers *a* and *b* into `a_r`, `b_r` (8 flip-flops with asynchronous active-low reset), forms the 16 partial-product bits pp_i[j] = a_r[j]·b_r[i], and adds them with three rows of four explicitly instantiated `full_adder`s (12 instances) wired exactly as in the connection table of the guide: row 0 is AND gates only, each later row is a 4-bit ripple-carry adder, the lowest sum of each row is a finished product bit and the other three sums plus the final carry pass down to the next row. The 8-bit result is registered into `p` (8 more flip-flops, 16 in total). The `*` operator is not used.

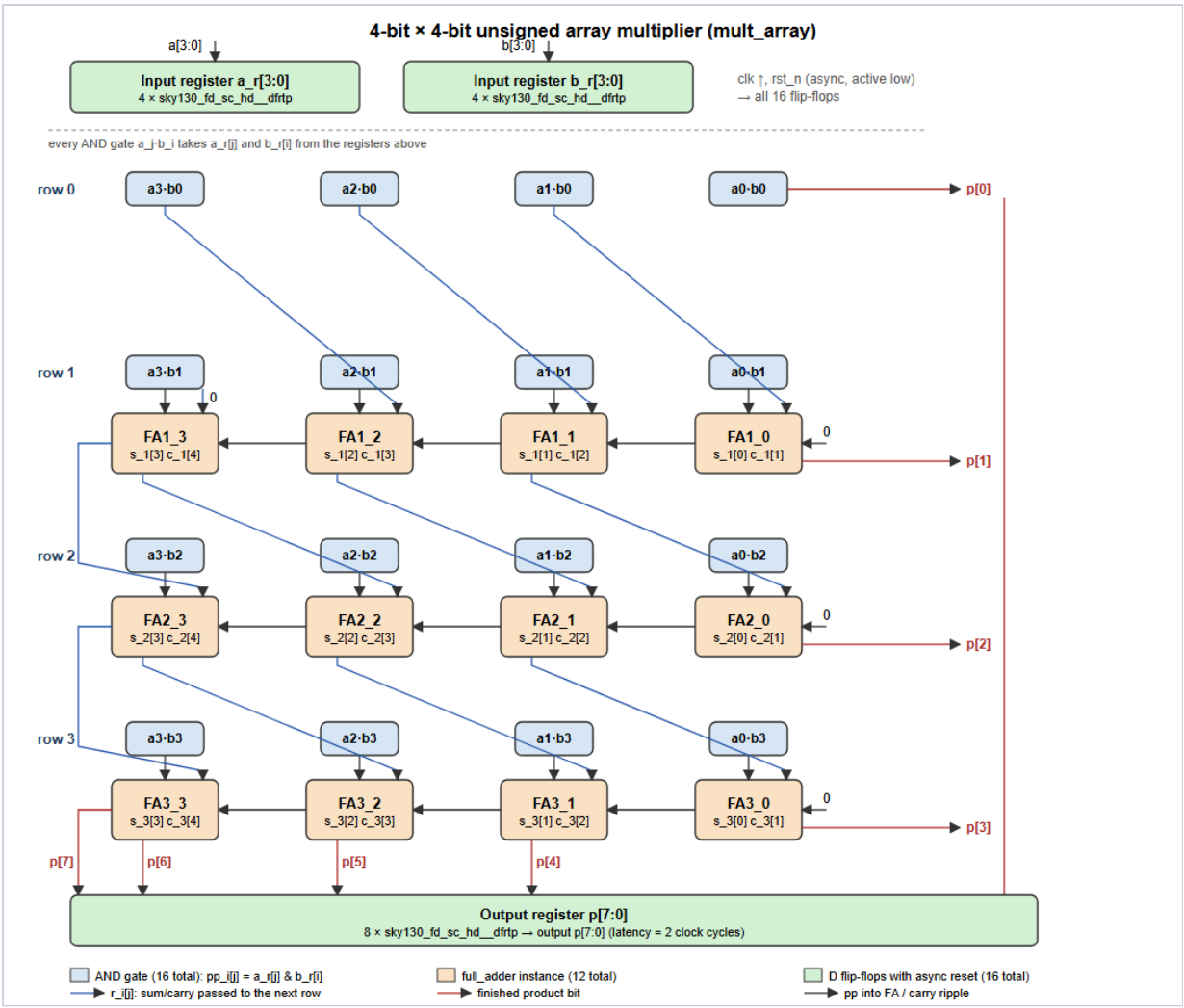


Figure 1 – Block diagram of mult_array: 16 AND gates (blue), 12 full adders in three ripple-carry rows (orange), 8 input and 8 output flip-flops (green). Blue wires carry r_i to the next row; red wires are finished product bits.

Testbench. `tb_mult` generates a 10 ns clock, holds `rst_n` low for the first two cycles, then applies all 256 (a, b) pairs. Each pair is applied on a falling edge, the testbench waits for two rising edges (the 2-cycle latency) and compares *p*

with $a \times b$ in the middle of the next cycle using `!==` so X/Z values also count as errors. Every mismatch is printed and one summary line ends the run; the waveform goes to `mult.vcd`.

Test	Result
RTL simulation (iverilog -g2012)	`PASS: 256 vectors checked, 0 errors`
Error injection: *sum* = *a* ^ *b* in full_adder.v	`FAIL: 256 vectors checked, 95 errors`
Gate-level netlist simulation (Part B)	`PASS: 256 vectors checked, 0 errors`

Table 1 – Simulation results. With the broken full adder the testbench reports the first mismatch at a = 3, b = 3 (p = 5, expected 9), so it detects real errors.

Hand-worked example. The last digit of my roll number is 0, so A = 0 + 5 = 5 and B = 7.

```
      0 1 0 1      A = 5
    x 0 1 1 1      B = 7
---------------
      0 1 0 1      pp0 = A AND b0 (b0 = 1)
    0 1 0 1        pp1 = A AND b1 (b1 = 1), shifted 1
  0 1 0 1          pp2 = A AND b2 (b2 = 1), shifted 2
0 0 0 0            pp3 = A AND b3 (b3 = 0), shifted 3
---------------
0 0 1 0 0 0 1 1      P = 35
```

Row	pp_i[3:0]	r_(i−1) [3:0]	s_i[3:0]	c_i[4:1]	product bit	r_i[3:0] passed down
0	0101	–	–	–	p[0] = 1	0010
1	0101	0010	0111	0000	p[1] = 1	0011
2	0101	0011	1000	0111	p[2] = 0	0100
3	0000	0100	0100	0000	p[3] = 0, p[7:4] = 0010	–

Table 2 – Values inside the array for 5 × 7, row by row (MSB first).

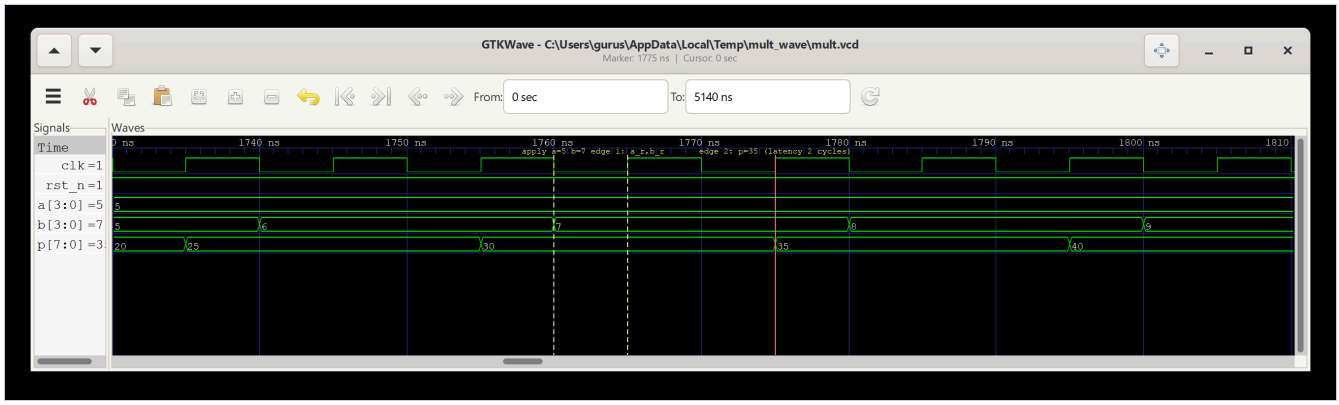


Figure 2 – GTKWave: a = 5 and b = 7 are applied at marker A, captured into a_r/b_r at rising edge 1 (marker B) and the product p = 35 appears after rising edge 2 (marker C): latency = 2 clock cycles = 20 ns at the 10 ns test clock.

3 Synthesis (Part B)

`synth/synth.tcl` runs the seven Yosys steps from the guide (`read_verilog`, `synth -top mult_array -flatten`, `dfflibmap`, `abc`, `opt_clean -purge`, `stat -liberty`, `write_verilog -noattr`) with the typical-corner Liberty file `sky130_fd_sc_hd__tt_025C_1v80.lib`.

Metric	Value
Total number of cells	70
Number of flip-flops	16 (sky130_fd_sc_hd__dfrtp_1)
Total cell area	797.014 µm²
Flip-flop area	16 × 25.024 µm² = 400.384 µm²
Flip-flop area (% of total)	50.24 %
3 most-used cell types	dfrtp_1 (16), nand2_1 (10), clkinv_1 (6)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 3 – Part B results (from synth/mult_array_stat.rpt).

Cell	Count	Area (µm²)
sky130_fd_sc_hd__dfrtp_1	16	400.384
sky130_fd_sc_hd__xnor3_1	3	67.565
sky130_fd_sc_hd__maj3_1	5	50.048
sky130_fd_sc_hd__nand2_1	10	37.536
sky130_fd_sc_hd__xnor2_1	3	26.275
sky130_fd_sc_hd__xor3_1	1	23.773
sky130_fd_sc_hd__clkinv_1	6	22.522
sky130_fd_sc_hd__a21oi_1	4	20.019
sky130_fd_sc_hd__and4_1	2	17.517
sky130_fd_sc_hd__a22oi_1	2	15.014
sky130_fd_sc_hd__o21ai_0	3	15.014
sky130_fd_sc_hd__and2_0	2	12.512
sky130_fd_sc_hd__and3_1	2	12.512
sky130_fd_sc_hd__a2111o_1	1	11.261
sky130_fd_sc_hd__nor2_1	3	11.261
sky130_fd_sc_hd__mux2i_1	1	10.010
sky130_fd_sc_hd__a22o_1	1	8.758
sky130_fd_sc_hd__o41ai_1	1	8.758
sky130_fd_sc_hd__xor2_1	1	8.758
sky130_fd_sc_hd__nand4_1	1	6.256
sky130_fd_sc_hd__nor2b_1	1	6.256
sky130_fd_sc_hd__nor3_1	1	5.005

Table 4 – All cell types after synthesis, by area.

4 Timing (Part C)

`sta/sta.tcl` reads the Liberty file and the netlist, defines `clk`, sets 1.0 ns input and output delays, prints the worst setup slack and saves the critical-path report. With T = 10 ns the worst slack is 7.017 ns, so T_min = 10 − 7.017 = 2.983 ns. Re-running with T = 2.98 ns gives a slack of -0.003 ns, i.e. zero to within the 0.01 ns rounding.

Metric	Value
Worst slack at T = 10 ns	7.017 ns
T_min	2.983 ns
F_max	335.2 MHz
Slack at T = 2.98 ns	-0.003 ns
Critical path start point	_109_ = a_r[1] register (clock pin)
Critical path end point	_107_ = p[7] register (D pin)
Cells on the critical path	8 logic cells (+ 2 flip-flops)
Latency = 2 × T_min	5.966 ns
Throughput	335.2 million results/s

Table 5 – Part C results (typical corner).

#	Instance	Cell	Incr (ns)	Arrival (ns)	Role
0	_109_	dfrtp_1	0.396	0.396	launch flip-flop a_r[1]
1	_054_	a22o_1	0.248	0.644	partial products a1·b1, a2·b0 (AND–OR)
2	_055_	and3_1	0.181	0.824	row-1 full-adder logic (carry)
3	_058_	nor3_1	0.276	1.101	row-1 full-adder logic (carry into row 2)
4	_070_	xnor3_1	0.453	1.554	row-2 full-adder sum (3-input XOR)
5	_071_	and3_1	0.209	1.763	pp a0·b3 and FA3_0 carry (half adder, c_3[0] = 0)
6	_085_	maj3_1	0.361	2.124	FA3_1 carry (majority)
7	_097_	maj3_1	0.379	2.503	FA3_2 carry (majority)
8	_098_	maj3_1	0.349	2.852	FA3_3 carry c_3[4] → p[7]
	_107_	dfrtp_1	-0.131	2.852	capture flip-flop p[7] (setup time)

Table 6 – Critical path at T = 10 ns, in order (from sta/mult_array_timing.rpt).

5 Physical design (Part D)

`pnr/config.json` sets DESIGN_NAME, VERILOG_FILES, CLOCK_PORT = clk, FP_SIZING = absolute and a fixed 150 µm × 150 µm DIE_AREA. The starting clock period was 1.2 × T_min = 3.58 ns, rounded up to 4.0 ns. LibreLane signs off timing at nine PVT corners; the top-level `timing__setup__ws` in metrics.json is the worst of them (the slow ss_100C_1v60 corner), so the period was raised by 0.5 ns per run until that worst-corner slack was ≥ 0 (Table 7).

Run	CLOCK_PERIOD (ns)	Setup slack, worst corner (ns)	Setup slack, tt corner (ns)	DRC	LVS
1	4.0	-1.254	1.167	0	0
2	4.5	-0.754	1.667	0	0
3	5.0	-0.254	2.167	0	0
4	5.5	-0.009	2.595	0	0
5	6.0	-0.001	2.929	0	0
6	6.5	0.499	3.429	0	0

Table 7 – Every LibreLane run (also recorded in README.md).

Metric	Value
CLOCK_PERIOD used	6.5 ns
Number of runs needed	6
Core area	17,759.5 µm² (die 22,500 µm²)
Cells after synthesis (Part B)	70
Cells after place and route	4,822 (336 excluding fill/decap)
Kinds of cells added	decap 4214, fill 272, tap 245, buffer 19, clock buffer 3
Wirelength	2,612 µm
Setup slack	+0.499 ns worst corner (tt: +3.429 ns)
Post-route F_max	166.6 MHz worst corner (tt: 325.6 MHz)
F_max from Part C	335.2 MHz
DRC errors	0 (Magic 0, KLayout 0, route 0)
LVS errors	0

Table 8 – Part D results (from pnr/metrics.json of the final run).

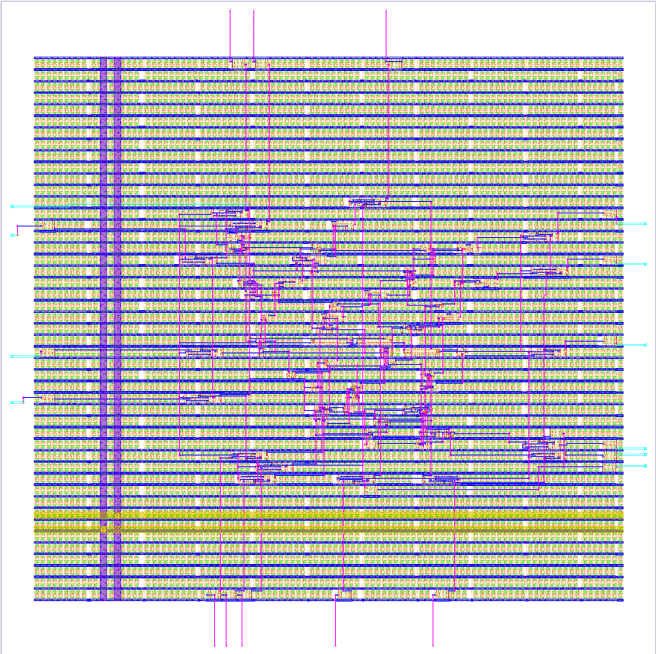


Figure 3 – Final layout, whole 150 µm × 150 µm die (KLayout).

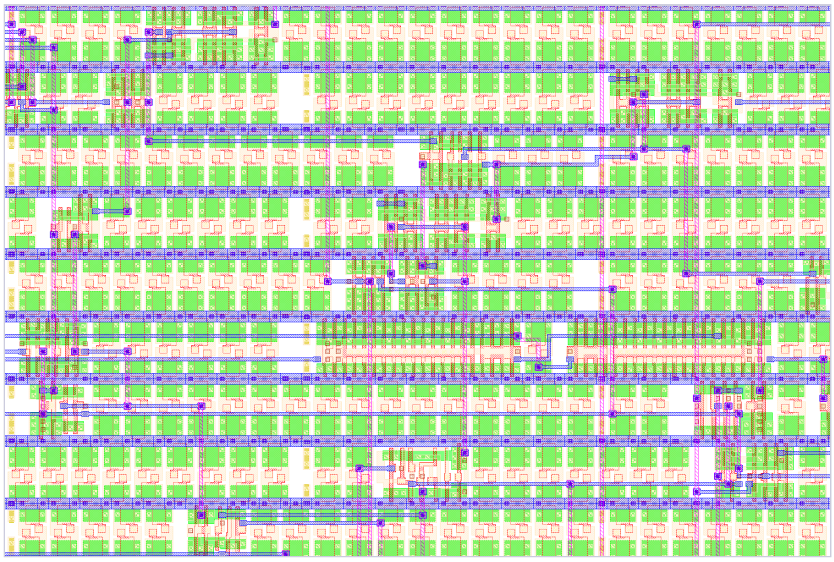


Figure 4 – Zoomed layout: standard-cell rows (horizontal power rails on met1) with routed met1/met2 wires and vias.

6 Answers to questions

Q1. Synthesis produced 70 cells, of which 16 are flip-flops, so 54 cells implement the logic – not 16 + 12 = 28. A full adder is not one cell here: ABC maps each sum to an XOR3/XNOR3 (or XOR2 chain) and each carry to a MAJ3 or an AOI/OAI combination, so one adder needs about two cells, and polarity fixing adds inverters (6 clkinv cells). In the other direction ABC also removes and merges logic because the design is flattened: the zero carry-ins (c_i[0] = 0) and r_0[3] = 0 turn four adders into half adders or wires, and many AND gates are absorbed into complex cells such as a22o, a21oi and a2111o. The count is therefore set by the cell library and by logic optimisation, not by the number of RTL modules.

Q2. The critical path starts at the clock pin of the `a_r[1]` input flip-flop and ends at the D pin of the `p[7]` output flip-flop (register to register). It has 8 logic cells (Table 6); data arrive at 2.852 ns, including 0.396 ns clock-to-Q, and the setup time is 0.131 ns. 7 of the 8 cells implement full-adder sum/carry logic and the remaining one is partial-product logic. The path runs through row 1, the row-2 sum and then the whole row-3 carry chain to c_3[4], close to the estimate of 2N = 8 full-adder delays. It is shorter in cells than eight full adders would be because single SKY130 cells (MAJ3 for a carry, XNOR3 for a sum) implement a whole full-adder output, and the half adders at the start of each row are faster than full adders. The average is (2.852 − 0.396) / 8 ≈ 0.307 ns per full-adder delay.

Q3. Flip-flops take 400.384 of 797.014 µm², i.e. 50.2 % of the cell area. One `dfrtp_1` is 25.024 µm², while a typical gate is 3.75–10 µm², because a flip-flop with asynchronous reset contains two latches, clock inverters and the reset logic (~25 transistors). The design has 16 flip-flops but only 54 small logic cells, so the registers dominate. Logic grows as N^2 (N^2 AND gates, N(N−1) full adders) while the registers only grow as 4N, so the flip-flop share falls for wider multipliers.

Q4. Part C used ideal wires, an ideal clock and only the typical corner. After routing, every net has extracted wire resistance and capacitance (total wirelength 2,612 µm), the real clock tree adds insertion delay and skew, and LibreLane signs off at nine corners including slow silicon at 100 °C and 1.60 V. In the worst (ss) corner the slack is +0.499 ns at 6.5 ns, so F_max = 1000 / (6.5 − 0.499) = 166.6 MHz, lower than the 335.2 MHz from Part C. Even at the same tt corner the post-route value, 325.6 MHz, is lower: besides the wire RC and the clock-tree skew, LibreLane adds a 0.25 ns clock uncertainty, the I/O buffers sit in the path, and its timing-driven synthesis chose smaller, slower cells for the relaxed 6.5 ns target. The added cells are: *tap* cells (245) that tie the n-well and substrate to the supplies to prevent latch-up; *decap* cells (4214) that hold charge between VPWR and VGND to reduce supply noise; *fill* cells (272) that close the gaps in each row so wells and power rails are continuous and density rules are met; *buffers* (19) for input/output isolation, fan-out and timing repair; and *clock buffers* (3) from clock-tree synthesis so all 16 flip-flops see the clock with small skew.

Q5. DRC (design rule check) verifies that the layout geometry obeys the foundry's manufacturing rules: minimum widths, spacings, enclosures, overlaps and densities on each layer. LVS (layout versus schematic) extracts the transistors and connectivity from the layout and checks that they form the same circuit as the gate-level netlist (same devices, nets and pins). DRC failures mean the chip cannot be printed reliably (shorts, opens, low yield); LVS failures mean the printed chip is not the designed circuit. Both must pass, as here (DRC 0, LVS 0), because either kind of error makes the silicon useless and a mask set is far too expensive to risk.

Q6. For N = 8: full adders N(N−1) = 8 × 7 = 56 instead of 12 (4.7×) and AND gates N^2 = 64 instead of 16 (4×). The critical path is about 2N = 16 full-adder delays instead of 8 (2×). Using my measured 0.307 ns per full-adder delay: 16 × 0.307 + 0.396 (clock-to-Q) + 0.131 (setup) ≈ 5.44 ns, so F_max ≈ 184 MHz compared with 335 MHz for N = 4. Area grows quadratically but delay only linearly with N.

7 Conclusion

The multiplier passes all 256 vectors at RTL and gate level, synthesises to 70 cells (797.0 μm², half of it flip-flops), meets T_min = 2.983 ns (335 MHz) in typical-corner STA, and is placed and routed at 6.5 ns with DRC = 0, LVS = 0 and non-negative setup slack in all nine corners. The measurements match the structure: about two cells per full adder, a critical path through the final ripple-carry row, and a layout whose cell count is dominated by fill, decap and tap cells rather than logic.

> **AI tool disclosure.** Claude (Anthropic) was used to help set up the toolchain, write the Verilog, Tcl and JSON files, run the flow and draft this report. I have reviewed every file and can explain each line.